

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук

Кафедра программирования и информационных технологий

Программа для создания скелетных анимаций

Курсовая работа по дисциплине

Теория информационных процессов и систем

6 семестр 2025/2026 учебного года

09.03.02 Информационные системы и технологии

Зав. кафедрой _____ д. ф.-м. н., доцент Махортов С. Д.

Обучающийся _____ ст. 3 курса Мышкевич А. Н.

Руководитель _____ ассистент Ушаков В. А.

Воронеж 2026

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ.....	4
ВВЕДЕНИЕ.....	5
1 Теоретическая часть.....	6
1.1 Анализ предметной области и актуальность.....	6
1.2 Обзор аналогов.....	6
1.2.1 Spine.....	6
1.2.2 LoongBones (DragonBones).....	7
1.2.3 Synfig Studio.....	8
1.3 Формат SpineJson.....	9
1.3.1 Skeleton (Скелет).....	9
1.3.2 Bones (Кости).....	10
1.3.3 Attachments (Привязки).....	11
1.3.4 Slots (Слоты).....	11
1.3.5 Skins (Скины).....	11
1.3.6 Animations (Анимации).....	13
1.3.7 Bone timeline (Типы дорожек костей).....	14
1.4 Интерполяция.....	14
2 Проектирование.....	16
2.1 Языки программирования и технологии.....	16
2.2 Функциональные и нефункциональные требования.....	16
2.3 Архитектура.....	19
2.3.1 MVVM.....	20
2.3.2 DI.....	21
2.3.3 Service и Common слои.....	21
3 Реализация.....	23
3.1 Реализация пользовательского интерфейса.....	23
3.2 Реализация ключевых функций.....	24
3.2.1 Управление скелетом.....	24

3.2.2 Работа со слотами.....	25
3.2.3 Анимация и таймлайн.....	26
3.2.4 Экспорт, импорт и управление файлами.....	28
3.2.5 Реализация нефункциональных требований.....	28
ЗАКЛЮЧЕНИЕ.....	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	33

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей курсовой работе применяют следующие термины с соответствующими определениями:

JSON (JavaScript Object Notation) — текстовый формат обмена данными, основанный на синтаксисе JavaScript. В данной работе используется для сохранения полной информации о скелете, спрайтах, таймлайне и ключевых кадрах анимации.

GIF (Graphics Interchange Format) — графический формат, поддерживающий анимированные изображения. Используется для экспорта анимации в виде циклически воспроизводимого видео без звука.

MP4 (MPEG-4 Part 14) — цифровой мультимедийный контейнерный формат для хранения видео, аудио и субтитров. В работе применяется для экспорта анимации в виде полноценного видеофайла.

PNG (Portable Network Graphics) — растровый формат хранения графической информации, поддерживающий прозрачность (альфа-канал). Используется для экспорта анимации покадрово в виде отдельных изображений.

JPG / JPEG (Joint Photographic Experts Group) — распространённый растровый формат хранения изображений с потерями. Применяется для экспорта кадров анимации с возможностью регулировки качества сжатия.

MVVM (Model-View-ViewModel) — распространённый архитектурный паттерн для UI с разделением логики и представления. Применяется для построения клиентских приложений с возможностью тестирования бизнес-логики без привязки к интерфейсу.

DI (Dependency Injection) — распространённый паттерн для передачи внешних зависимостей компоненту вместо их внутреннего создания. Применяется для снижения связанности кода с возможностью замены реализаций без изменения существующих классов.

ВВЕДЕНИЕ

В современном мире анимация приобретает всё более высокую востребованность, популярность и необходимость. Она активно применяется не только в развлекательной индустрии, но также в рекламной сфере и во множестве других областей. Вследствие этого особую актуальность обретает задача оптимизации процесса создания анимации: снижения его трудоёмкости и ресурсоёмкости при одновременном повышении эффективности.

Анимация представляет собой последовательность сгенерированных изображений (кадров). На ранних этапах развития технологий каждый кадр приходилось отрисовывать вручную, что требовало значительных временных затрат. Появление скелетной анимации кардинально изменило сложившуюся ситуацию.

Скелетная анимация — это метод, при котором изображение разделяется на спрайты, привязанные к иерархической структуре костей. В ключевых позициях задаётся расположение костей, а их промежуточные положения между ключевыми кадрами вычисляются с помощью интерполяции, благодаря чему осуществляется движение спрайтов. То есть, Основная идея заключается в создании структуры скелета, состоящей из костей, которые связаны в иерархическую структуру и оказывают влияние на различные части объектов [4].

Данный подход позволяет существенно сократить издержки на создание анимации. Более того, анимация может воспроизводиться непосредственно в ходе выполнения программы на различных ЭВМ, что обеспечивает экономию дискового пространства: вместо хранения последовательности растровых изображений достаточно сохранить лишь набор спрайтов и информацию о ключевых кадрах. Эти преимущества обуславливают ещё более высокую востребованность скелетной анимации.

1 Теоретическая часть

1.1 Анализ предметной области и актуальность

Предметной областью является создание и редактирование скелетных анимаций. Необходимо выделить следующие процессы:

- Возможность создавать кости и поддерживать их иерархию.
- Возможность перемещать спрайты и связывать их с костями.
- Интерполировать координаты объектов между ключевыми кадрами с учетом их иерархии.
- Возможности переименования и удаления объектов.
- Предпросмотр анимации в процессе работы над ней.
- Экспорт анимации в виде последовательности картинок или целого видео.
- Экспорт и импорт анимации в JSON формате.
- Возможность редактировать получившийся json файл.

Актуальность обуславливается ростом востребованности скелетной анимации в современном мире и небольшим количеством подходящих аналогов.

1.2 Обзор аналогов

1.2.1 Spine

Эта программа является наиболее популярной среди всех аналогичных продуктов. Spine имеет множество функций, позволяет экспортировать анимацию в формате json, что дает возможность использовать эти анимации в других программах. Имеет временную шкалу, может делить спрайты на сетки, графы, веса, ограничивающие рамки, свободные деформации и много других функций [6].

Но, к сожалению, есть значительные минусы. Во-первых, программа не имеет бесплатной версии. В пробной бесплатной версии просто нет функции экспорта. Также программа доступна для покупки только на официальном сайте, что делает ее недоступной для многих регионов.

Рисунок 1 демонстрирует пользовательский интерфейс бесплатной пробной версии продукта.

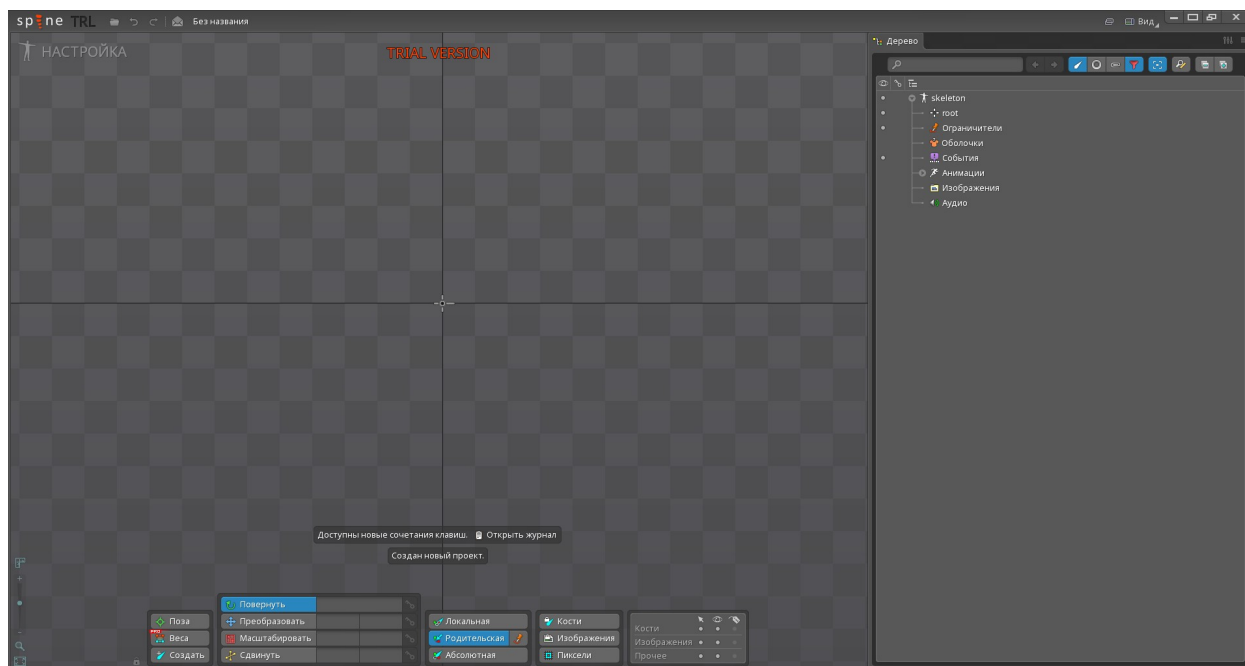


Рисунок 1 — Spine

На нем видно, что проверить и оценить функции возможно, но экспортировать работу никак не получится.

1.2.2 LoongBones (DragonBones)

Это бесплатная программа, имеет открытый исходный код. Она также предоставляет функции весов, свободной деформации, сеток, ограничительные рамки и многое другое [2].

Но есть и значительные недостатки. Сейчас доступна только веб-версия приложения, из-за чего для работы необходим интернет, а также не во всех регионах сайт доступен. Чтобы использовать, необходима регистрация, что также является минусом.

Рисунок 2 демонстрирует пользовательский интерфейс программного продукта.

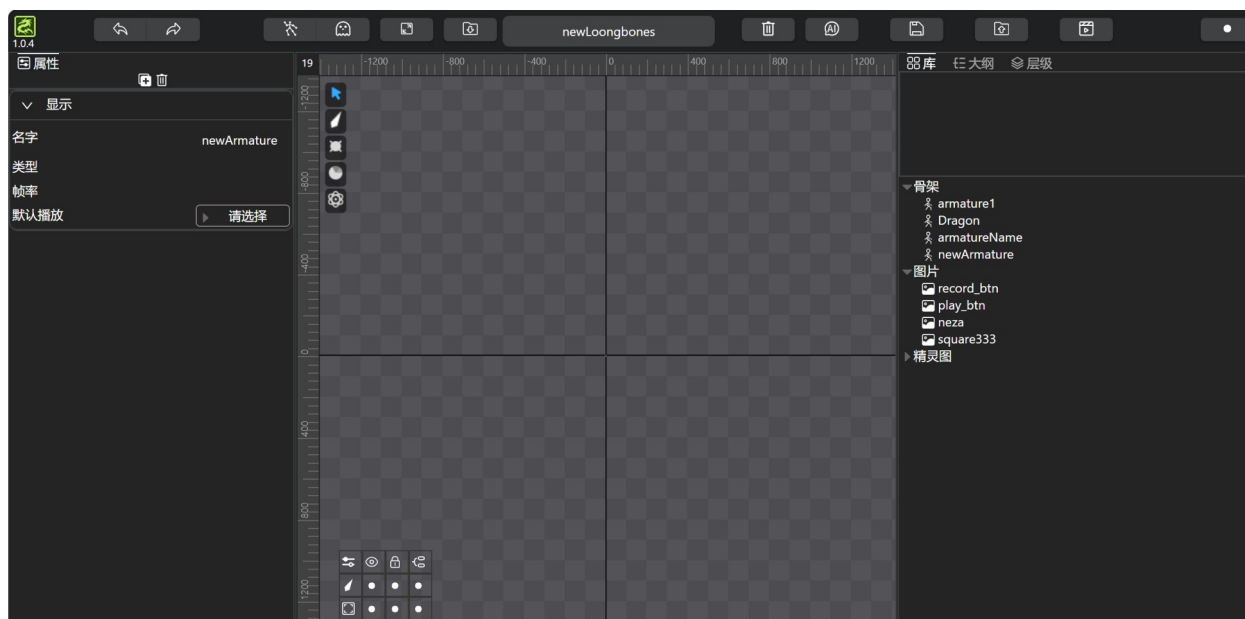


Рисунок 2 — LoongBones (ранее DragonBones)

Можно заметить, что интерфейс похож на Spine.

1.2.3 Synfig Studio

Это еще один бесплатный программный продукт с открытым исходным кодом [3]. Он обладает достаточно большим функционалом, например, позволяет создать скелет и его иерархию, привязывать картинки [3].

Но в отличие от предыдущих программ, он не специализируется на скелетных анимациях, поэтому функций в нем меньше. Он не имеет

возможности экспортировать анимации в json формате, что значительно ограничивает возможности его применения [3].

Рисунок 3 демонстрирует главный экран программного продукта.

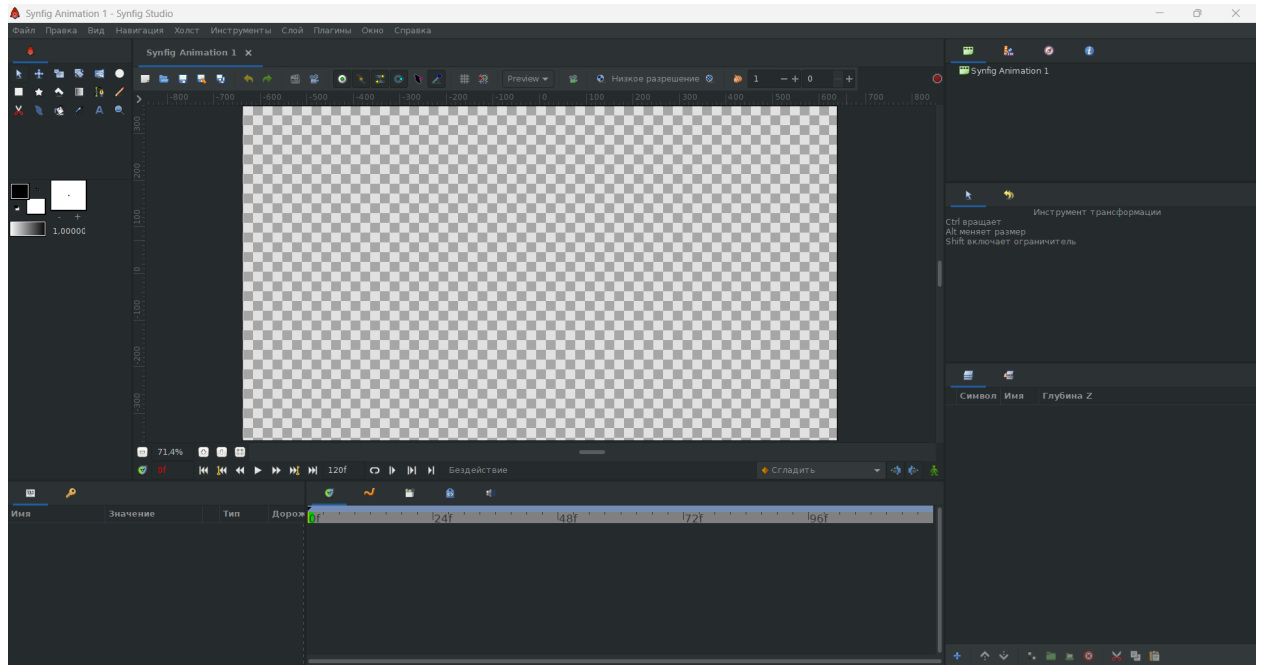


Рисунок 3 — SynfigAnimation

Интерфейс почти полностью повторяет аналогичные программы.

1.3 Формат SpineJson

Благодаря популярности Spine именно этот формат стал стандартом для скелетных анимаций. Это простой и удобный для чтения формат.

В данном продукте были использованы следующие секции SpineJson:

1.3.1 Skeleton (Скелет)

Эта секция описывает весь скелет, его метаданные.

Ее структура:

```

"skeleton": {
    "hash": "5WtEfO08B0TzTg2mDqj4IHypUZ4",
    "spine": "3.8.24",
    "x": -17.2,
    "y": -13.3,
    "width": 470.86,
    "height": 731.44,
    "images": "./images/",
    "audio": "./audio/"
},

```

Листинг 1 — Секция Skeleton [1].

В данной продукте используется только поле версии Spine. Это поле необходимо в каждом SpineJson файле для корректной работы других программ, в которые нужно поместить анимацию.

1.3.2 Bones (Кости)

Это одна из ключевых секций файла. Она описывает все кости и их базовую позу. Базовая поза — это смещение каждой кости относительно скелета. Это необходимо для корректной загрузки костей из файла. Важно, что в анимации эти значения могут не использоваться, но при этом необходимы.

```

"bones": [
    { "name": "root" },
    { "name": "torso", "parent": "root", "length":
85.82, "x": -6.42, "y": 1.97, "rotation": 94.95 },
    ...

```

```
],
```

Листинг 2 — Пример секции костей [1].

В данной работе задействованы все поля костей.

1.3.3 Attachments (Привязки)

Привязки — это видимые в итоговой анимации элементы, например, картинки. Существует большое количество типов привязок. В данном программном продукте реализованы только изображения.

1.3.4 Slots (Слоты)

Важной частью анимации являются слоты. Слоты — это модификации, прикрепляемые к кости. У одной кости может быть сколько угодно слотов. Слоты могут осуществлять связь с разными типами привязок (Attachment). В данной работе реализована только привязка изображения.

```
"slots": [  
  { "name": "left shoulder", "bone": "left  
  shoulder", "attachment": "left-shoulder" },  
  { "name": "left arm", "bone": "left arm",  
  "attachment": "left-arm" },  
  ...  
],
```

Листинг 3 — Секция Slots [1].

В программном продукте реализованы все поля слота.

1.3.5 Skins (Скины)

Скины — это наборы привязок, которые можно подменять друг на друга. Если привязка принадлежит к скину, который не активен в данный момент, то она не показывается, а слот выглядит пустым.

```
"skins": [  
  {  
    "name": "skinName",  
    "attachments": {  
      "slotName": {  
        "attachmentName": { "x": -4.83, "y":  
10.62, "width": 63, "height": 47 },  
        ...  
      },  
      ...  
    }  
  },  
  {  
    "name": "skinName",  
    "attachments": {  
      "slotName": {  
        "attachmentName": { "name":  
"actualAttachmentName", "x": 53.94, "y": -5.75,  
"rotation": -86.9, "width": 121, "height": 132 },  
        ...  
      },  
      ...  
    }  
  },  
  ...  
]
```

```
],
```

Листинг 4 — Секция Skins [1].

В данном продукте именно в скинах хранятся списки привязок и их поля.

1.3.6 Animations (Анимации)

Эта секция описывает трансформации костей во времени.

```
"animations": {
  "name": {
    "bones": { ... },
    "slots": { ... },
    "ik": { ... },
    "deform": { ... },
    "events": { ... },
    "draworder": { ... },
  },
  ...
}
```

Листинг 5 — Секция Animations [1].

Эта секция содержит список участвующих в движении костей и их трансформаций. Важно отметить и то, что эта секция содержит draworder — эта секция связывает слоты и их порядок отрисовки, то есть помогает управлять видимостью слотов во время анимации.

В данном приложении используются только секции bones и draworder.

1.3.7 Bone timeline (Типы дорожек костей)

У костей есть разные типы трансформаций, которые описываются дорожками.

```
{
  "bones": {
    "boneName": {
      "timelineType": [
        { "time": 0, "value": -26.55 },
        { "time": 0.1333, "value": -8.78 },
        ...
      ],
      ...
    },
    ...
  },
  ...
}
```

Листинг 6 — Секция дорожек костей [1].

В данной работе применяются два типа дорожек — перемещение по X и Y и поворот в градусах.

1.4 Интерполяция

Скелетная анимация имеет преимущества за счет того, что позволяет не рисовать промежуточные кадры. Это достигается при помощи интерполяции.

Интерполяция — это нахождение неизвестных промежуточных значений с помощью нескольких известных [5].

В данной работе используется только линейная интерполяция, хотя существует и много других вариантов. Линейная интерполяция — это метод

нахождения промежуточных значений функции между двумя известными точками, предполагающий, что данные изменяются равномерно по прямой линии.

Формула линейной интерполяции представлена ниже.

$$y = y_1 + (x - x_1) \frac{(y_2 - y_1)}{(x_2 - x_1)}$$

Интерполяция является одним из важнейших свойств скелетной анимации и незаменима для ее работы.

2 Проектирование

2.1 Языки программирования и технологии

Для реализации программного продукта были выбраны язык C# и фреймворк Avalonia UI.

C# был выбран из-за его кроссплатформенности, большого количества доступных библиотек, высокой производительности, а также болееудобной работой с памятью по сравнению с C/C++.

Avalonia UI была выбрана потому, что не зависит от Windows и позволяет создавать программы и для других платформ. Также она дает возможность создавать красивый дизайн и напоминает CSS.

В качестве основы для темы была выбрана библиотека SukiUI. Она поддерживает и светлую, и темную темы, а также позволяет создавать свои темы. Более того, она содержит много улучшенных элементов управления, например, для уведомлений.

Для работы с JSON файлами была выбрана библиотека Newtonsoft.Json. Она позволяет удобно работать с файлами разной структуры.

Изображения для кнопок были взяты из библиотеки IconPacks.Avalonia.BootstrapIcons.

Для экспорта изображений в формате картинок или видео используется библиотека SixLabors.ImageSharp. Для создания MP4 файла используется ffmpeg.exe, который пользователи могут установить отдельно.

2.2 Функциональные и нефункциональные требования

В ходе анализа предметной области были сформулированы следующие функциональные требования, описывающие взаимодействие пользователя с интерфейсом калькулятора и ожидаемые результаты:

- Система должна предоставлять возможность создания новой кости с уникальным именем по умолчанию.
- Система должна позволять устанавливать родительскую связь между костями с помощью кнопки.
- Система должна позволять переименовывать любую кость, спрайт или ресурс.
- Система должна позволять перемещать спрайты в рабочей области с помощью мыши.
- Система должна позволять привязывать выбранный спрайт к конкретной кости, после чего спрайт наследует её трансформации.
- Система должна позволять добавлять, удалять и редактировать ключевые кадры на временной шкале для позиции, поворота и масштаба.
- Система должна автоматически интерполировать координаты и повороты объектов между двумя ключевыми кадрами.
- Во время редактирования анимации система должна воспроизводить её в реальном времени (предпросмотр) с возможностью остановки, паузы.
- Система должна экспортировать анимацию в виде последовательности PNG-изображений или JPG-изображений (каждый кадр — отдельный файл).
- Система должна экспортировать анимацию в видеоформаты MP4 и GIF.
- Система должна сохранять полные данные анимации (скелет + спрайты + таймлайн) в формате JSON.
- Система должна управлять файлами — ресурсами проекта, настройками — находящимися на диске.

- Система должна поддерживать создание и открытие новых проектов.

Помимо функциональных, есть еще и нефункциональные требования:

- Система должна поддерживать анимацию с частотой 60 FPS.
- При поврежденном JSON-файле система должна выводить сообщение об ошибке и указать строку с некорректными данными.
- Временная шкала должна позволять масштабирование.
- Холст с анимацией должен поддерживать масштабирование.
- Программа должна иметь возможность переключения между русским и английским языком.
- Система должна иметь возможность переключать темы с темной на светлую и наоборот.

На рисунке 4 представлена диаграмма сценариев использования:

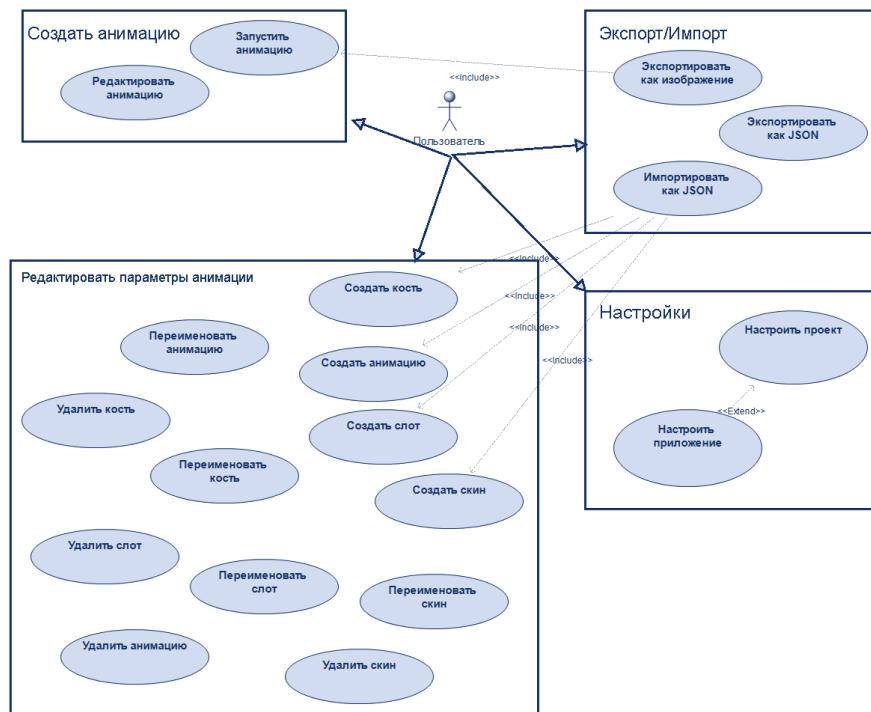


Рисунок 4 — Диаграмма сценариев использования

Она отображает основные функции приложения.

2.3 Архитектура

В программном продукте используется многослойная архитектура на основе MVVM. Связь View и ViewModel реализована через механизм привязок данных (Data Binding) и интерфейс InotifyPropertyChanged.

Также применяется внедрение зависимостей. Все сервисы (работа со скелетом, экспорт в JSON, сериализация) регистрируются в DI-контейнере и передаются в ViewModel через конструктор.

Полный список слоев:

- Models – содержит данные предметной области.
- Views – содержит пользовательский интерфейс.
- ViewModels – реализует связь между интерфейсом и данными предметной области.
- Services – содержит бизнес-логику, каждый сервис отвечает за свои действия.
- Common – вспомогательные методы, сервисы для обмена данными, конвертеры и другие объекты, к которым требуется доступ из многих мест программы.

На рисунке 5 изображена диаграмма деятельности.

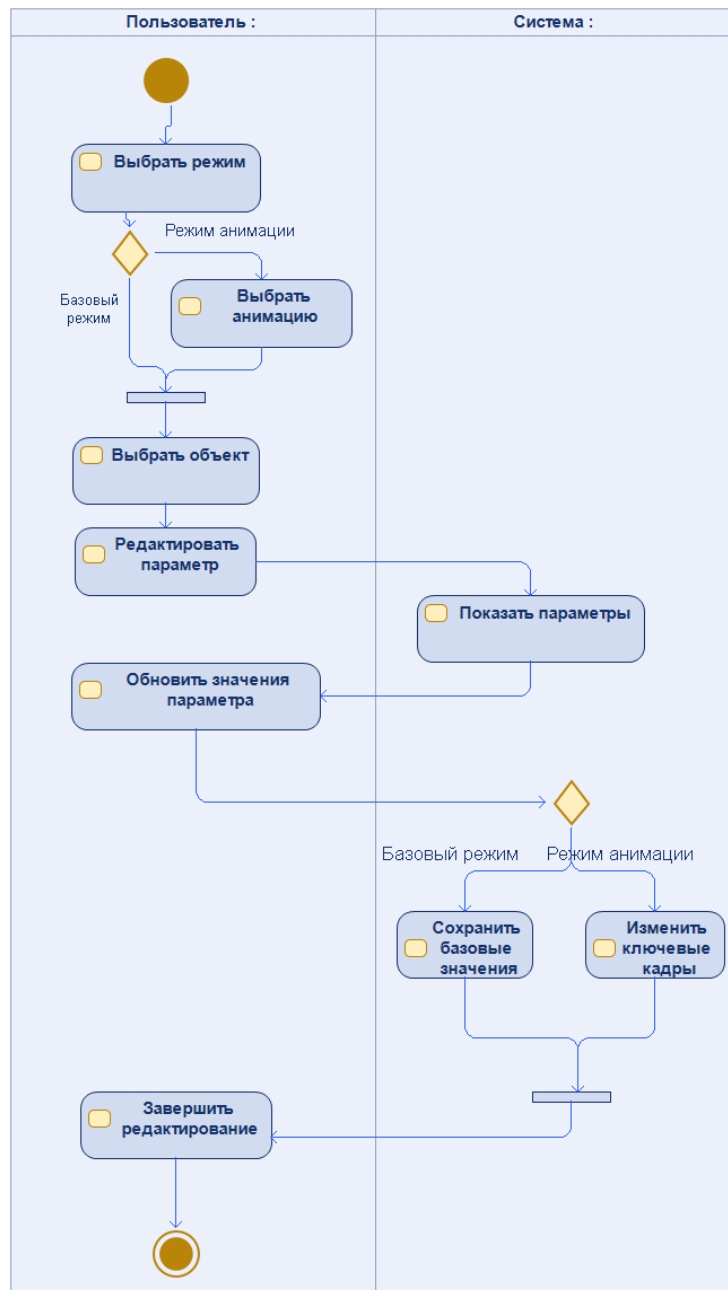


Рисунок 5 — Диаграмма деятельности

Она отображает последовательность работы в программе.

2.3.1 MVVM

Это один из наиболее популярных паттернов в современном мире, Avalonia UI работает именно с ним. Этот паттерн очень удобен, так как позволяет отделить логику приложения от визуальной части (представления).

Данный паттерн является архитектурным, то есть он задает общую архитектуру приложения [7]. Такой подход позволяет тестировать логику независимо и разделить ответственность.

Привязка данных осуществляется через Data Binding. Также используются динамические ресурсы для смены локализации программы.

View содержит только разметку и привязанные свойства/команды. Есть и немного кода в code-behind, например привязка некоторых горячих клавиш и события Drag-and-Drop, а также нажатия на главный Canvas.

ViewModel передает состояние приложения (текущий скелет, выбранную кость, таймлайн) во View.

Model представляет данные и методы предметной области (кости, спрайты, анимации) и не зависит от платформы.

2.3.2 DI

Используется внедрение зависимостей – идея работать с зависимостями вне зависимого класса [8]. Зависимости передаются в ViewModel через конструктор, а не создаются внутри неё. Это очень удобно, так как все зависимости регистрируются в едином месте.

Также если сервису нужны какие-то данные, они заполняются один раз при создании, а не каждый раз передаются в конструктор нового объекта сервиса.

Важно отметить, что все ViewModel также регистрируются как сервисы, что дает такие же преимущества.

2.3.3 Service и Common слои

Все сервисы содержатся в пространстве имен Service. Каждый сервис отвечает только за определенные действия. Они используются в разных частях приложения.

Common содержит классы и сервисы, которые не связаны с определенной предметной областью. Они хранят константы приложения, передают данные между разными ViewModel.

На рисунке 6 представлена диаграмма классов:

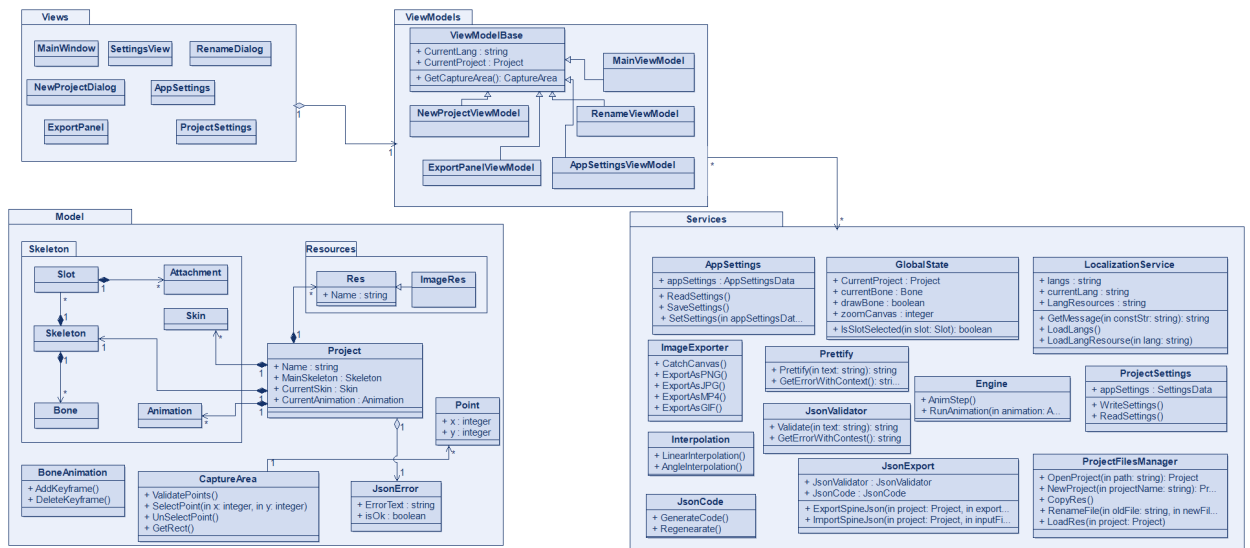


Рисунок 6 — Диаграмма классов

Она отображает основные связи между классами и пакетами.

3 Реализация

3.1 Реализация пользовательского интерфейса

На рисунке 7 представлен главный экран приложения. В центре находится основной холст — это рабочая область, где отрисовываются кости и анимация. Снизу находится таймлайн. Справа — иерархия скелета и детальная информация о выбранной кости. Слева — информация о скинах, анимациях и ресурсы (изображения), загруженные в проект.

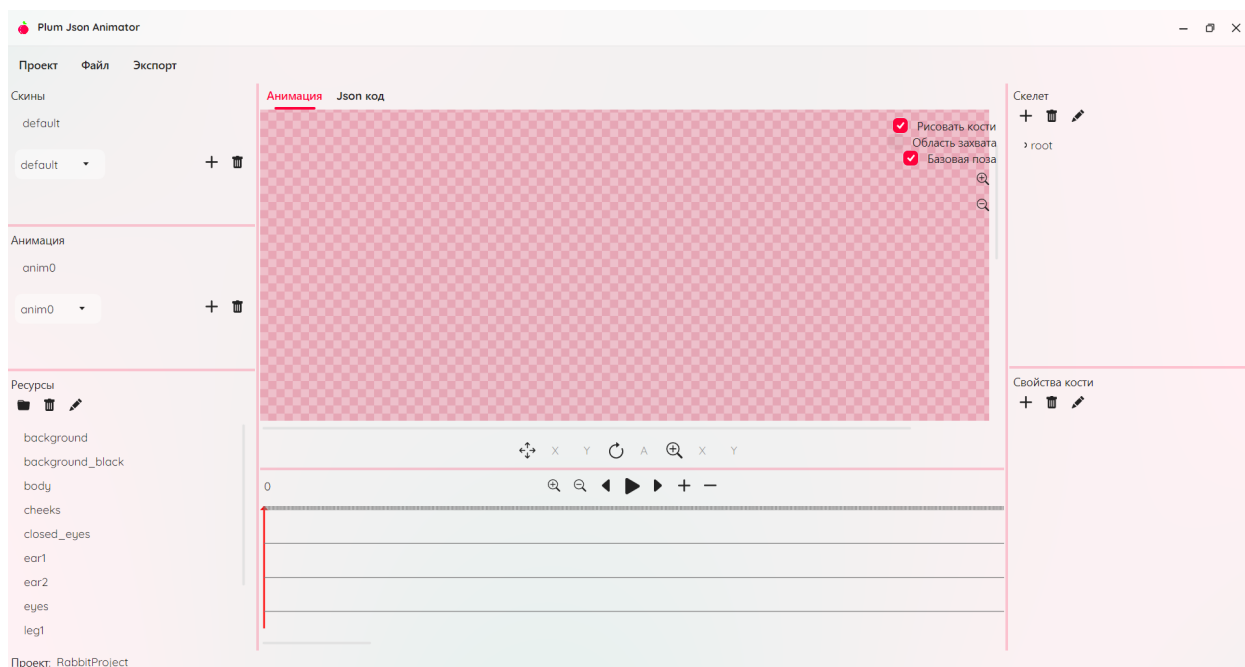


Рисунок 7 — Главный экран.

Также есть верхнее меню. Снизу отображается название текущего проекта.

На рисунке 8 можно увидеть вкладку для работы с JSON-кодом проекта. Здесь в центре текстовое поле, в котором отображается код, а снизу панель, показывающая информацию об ошибках в коде.

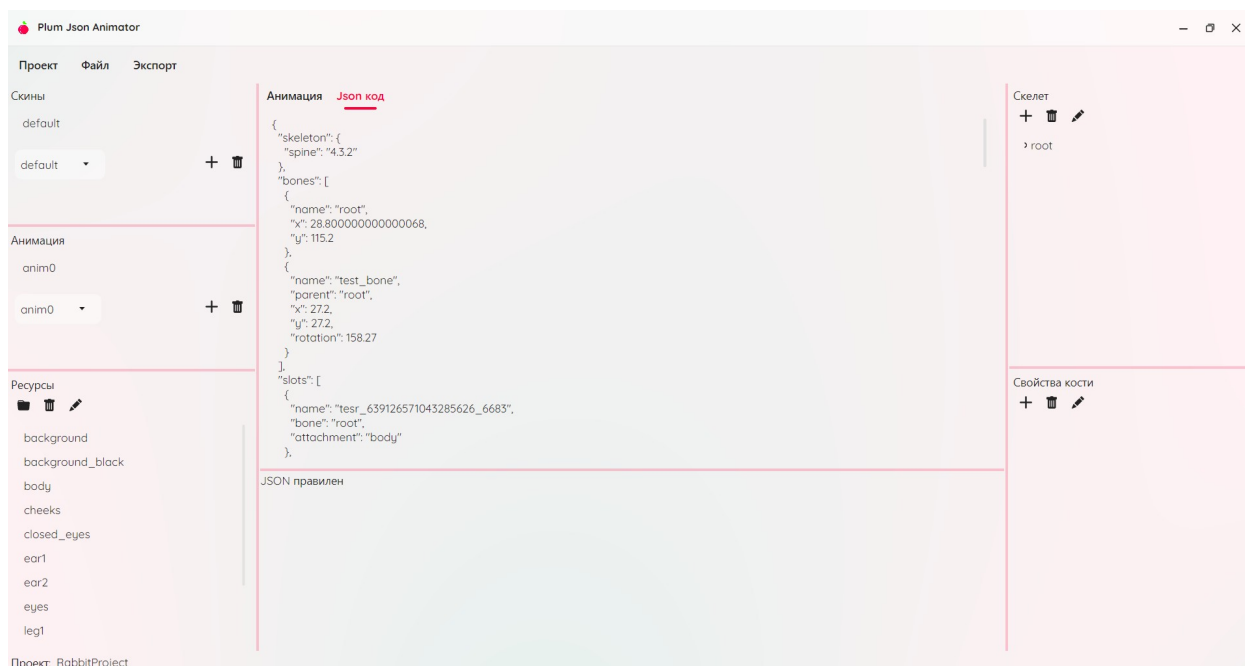


Рисунок 8 — Экран редактирования JSON

Если в JSON допущена ошибка, программа не будет генерировать проект из JSON и не даст переключиться на вкладку анимации, пока проблема не будет исправлена.

3.2 Реализация ключевых функций

3.2.1 Управление скелетом

Для работы со скелетом реализован следующий функционал:

- Создание кости. При нажатии кнопки система создаёт новый объект с автоматическим именем. Уникальность имени поддерживается при помощи комбинации текущей даты и случайного числа в имени.
- Иерархия костей. Любая добавленная кость становится потомком выбранной.
- Переименование. Любой объект (кость, спрайт, ресурс) можно переименовать специальной кнопкой.

3.2.2 Работа со слотами

- Перемещение спрайтов. Спрайты перемещаются по рабочей области с помощью мыши (drag-and-drop). При перемещении система обновляет их координаты в реальном времени.
- Привязка к кости. Пользователь перетаскивает ресурс на кость, после этого появляется слот, обладающий этой привязкой. После этого слот наследует все трансформации (положение, поворот, масштаб) родительской кости. При движении кости привязанный слот перемещается вместе с ней.

На рисунке 9 представлена иерархия костей и привязанный слот.

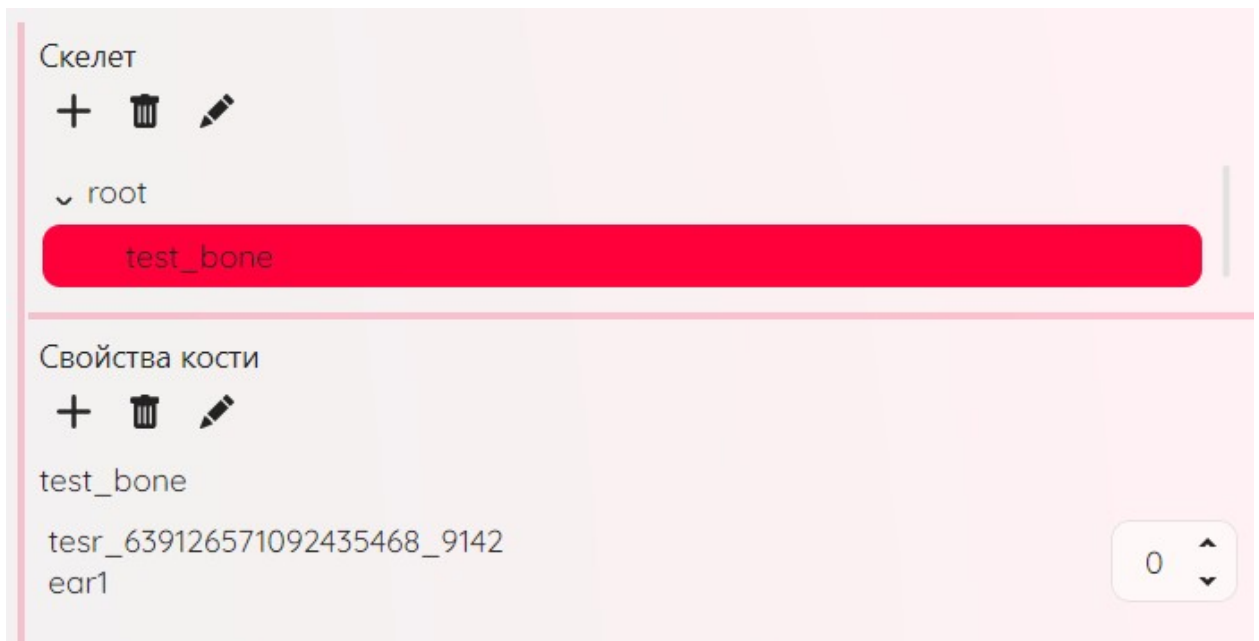


Рисунок 9 — Выбранная кость и ее слоты

На слот также можно нажать и увидеть его свойства и получить возможность перемещать его и трансформировать в координатной системе его кости.

В листинге 7 представлена разметка дерева костей.

```

<TreeView
    Name="boneTreeView"
    Grid.Row="2"
    Margin="0,5"
    ItemsSource="{Binding
CurrentProject.MainSkeleton.RootBones}">
    <TreeView.ItemTemplate>
        <TreeDataTemplate ItemsSource="{Binding
Children}">
            <StackPanel
DoubleTapped="OnBonePointerPressed">
                <TextBlock Text="{Binding Name,
Mode=TwoWay}" />
            </StackPanel>
        </TreeDataTemplate>
    </TreeView.ItemTemplate>
</TreeView>

```

Листинг 7 — Дерево костей

В основе дерева лежит элемент управления `TreeView`, каждая кость

3.2.3 Анимация и таймлайн

Реализована временная шкала (таймлайн), на которой пользователь может добавлять и удалять ключевые кадры. Для каждого кадра можно задать два параметра: позиция (X, Y) и поворот в градусах (угол).

Можно менять масштаб шкалы, видеть время, переключаться между кадрами, удалять и добавлять их.

На рисунке 10 можно увидеть временную шкалу с ключевыми кадрами.

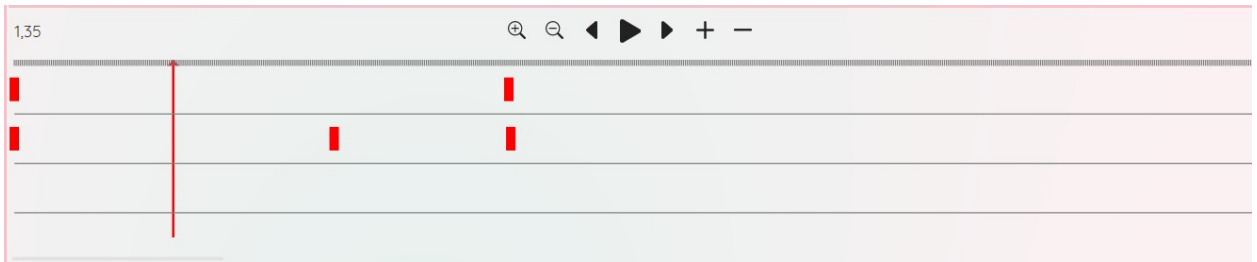


Рисунок 10 — Временная шкала (таймлайн)

При движении анимации положение каждой кости интерполируется между соседними ключевыми кадрами. За это отвечает отдельный сервис `Interpolation`, который применяет линейную интерполяцию ко заданным величинам.

```
public class Interpolation
{
    public double linearInterpolation(double
start, double end, double t)
    {
        return start + t * (end - start);
    }
    public double angleInterpolation(double
start, double end, double t)
    {
        return start + ((end - start + 540) %
360 - 180) * t;
    }
    public double findInterpolateParam(double
segmentDuration, double timeElapsed)
    {
```

```

        double t = segmentDuration > 0 ?
timeElapsed / segmentDuration : 1.0;
        return Math.Clamp(t, 0.0, 1.0);
    }
}

```

Листинг 8 — Сервис интерполяции

Предпросмотр и запуск анимации происходит на главном холсте. Сразу же можно и редактировать анимацию.

3.2.4 Экспорт, импорт и управление файлами

Экспорт анимации возможен несколькими способами:

- PNG/JPG – каждый кадр сохраняется как отдельный файл в выбранную папку.
- MP4/GIF – анимация экспортируется в видеоформат.

За это отвечает сервис ImageExporter.

За экспорт и импорт проекта в формате JSON отвечают уже другой сервис — JsonExport.

Система также имеет сервисы ProjectManager, который управляет проектами и файлами в них, ProjectSettings, который управляет сохранением и загрузкой настроек отдельного проекта из файла и AppSettings – сервис, который управляет глобальными настройками приложения.

3.2.5 Реализация нефункциональных требований

В ходе работы были реализованы следующие функциональные требования:

- Анимация имеет 60 FPS.
- Обработка ошибок JSON. При загрузке повреждённого JSON-файла система не падает, а выводит модальное окно с текстом

ошибки и номером строки. Пользователь может исправить файл вручную или выбрать другой.

- Временная шкала и холст масштабируются при помощи кнопок.
- Локализация. Программа поддерживает два языка: русский и английский. Переключение происходит через меню «Настройки» без перезапуска приложения. Все надписи, сообщения и подсказки динамически заменяются через механизм ресурсов.
- Также доступны 2 темы: темная и светлая.

На рисунке 11 можно увидеть главный экран приложения, но с темной темой и английским языком.

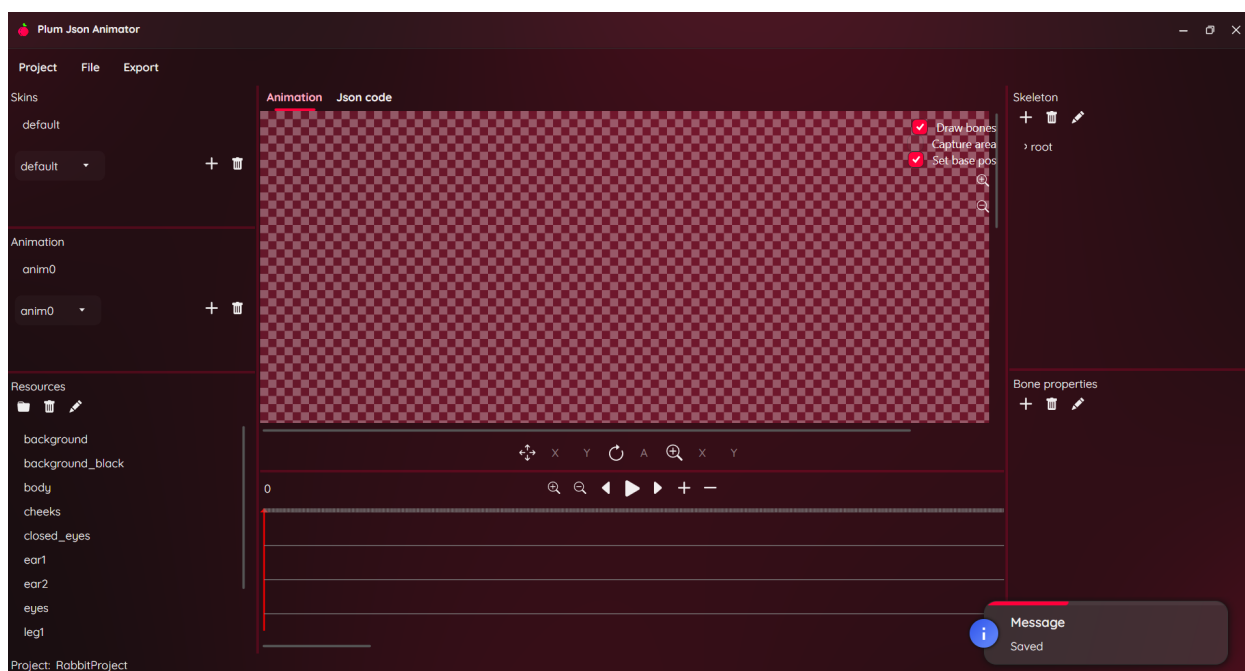


Рисунок 11 — Главный экран с темной темой и английским языком

Локализация осуществляется при помощи сервиса `LocalizationService`. Он работает со словарем ресурсов и предоставляет нужный перевод по ключу. Два базовых языка копируются в директорию программы на диске пользователя во время первого запуска. Полезной функцией является то, что пользователи самостоятельно могут добавить языки в программу, просто

изменив файлы исходных языков на нужные и скопировав их в директорию программы. В листинге 9 показан код копирования файлов языков.

```
private void CopyDefaultTranslations()
{
    var assembly =
Assembly.GetExecutingAssembly();
    var resourceNames = assembly
        .GetManifestResourceNames()
        .Where(name =>
name.EndsWith(".json") &&
name.Contains("Translations"));

    foreach (var resourceName in
resourceNames)
    {
        var fileName = resourceName
            .Replace("PlumJsonAnimator.Transla
tions.", "")
            .Replace("PlumJsonAnimator.", "")
            .TrimStart('.');
        using var stream =
assembly.GetManifestResourceStream(resourceName);
        if (stream != null)
        {
            using var reader = new
StreamReader(stream);
            string content =
reader.ReadToEnd();
```

```

        string destPath =
Path.Combine(LocalizationFilePath, fileName);
        File.WriteAllText(destPath,
content);
    }
}
}

```

Листинг 9 — Метод копирования файлов языков

Метод переносит заранее заготовленные файлы в папку приложения, если там их еще нет.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был разработан редактор скелетной анимации, позволяющий создавать и редактировать иерархические скелеты, привязывать спрайты к костям, настраивать ключевые кадры и экспортировать результат в различные форматы.

В теоретической части был проведён анализ предметной области, рассмотрены существующие аналоги (Spine, DragonBones, Synfig Studio), изучен формат Spine JSON и методы интерполяции. На основе этого анализа были сформулированы функциональные и нефункциональные требования к системе.

В практической части спроектирована архитектура приложения на базе паттернов MVVM и DI, выделен сервисный слой. Реализованы основные функции: управление скелетом и спрайтами, анимация по ключевым кадрам с интерполяцией, экспорт в PNG, MP4, GIF и JSON, а также поддержка проектов и локализация интерфейса.

Все поставленные функциональные требования выполнены, нефункциональные требования также реализованы.

В будущем система может быть расширена и улучшена: можно добавить разные варианты интерполяции, реализовать mesh, добавить больше вариантов привязок.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Esoteric Software LLC : [сайт]. – Франкфурт, 2013. – URL: <https://ru.esotericsoftware.com/spine-json-format> (дата обращения: 25.04.2026). – Текст. Изображение : электронные.
2. LoongBones : [сайт]. – Торонто, 2024. – URL: <https://www.loongbones.app/> (дата обращения: 31.05.2026). – Текст. Изображение : электронные.
3. Synfig Studio : [сайт]. – Санкт-Петербург, 2005. – URL: <https://docs.synfig.ru/> (дата обращения: 31.05.2026). – Текст. Изображение : электронные.
4. Уголок Ковалева : [сайт]. – Санкт-Петербург, 2020. – URL: <https://rekovalev.site/opengl-21-gltf2-skeletal-animation/> (дата обращения: 31.05.2026). – Текст. Изображение : электронные.
5. SkillFactory Media : [сайт]. – Москва, 2012. – URL: <https://blog.skillfactory.ru/approksimatsiya-interpolyatsiya-ekstrapolyatsiya/> (дата обращения: 31.05.2026). – Текст. Изображение : электронные.
6. Esoteric Software LLC : [сайт]. – Франкфурт, 2013. – URL: <https://ru.esotericsoftware.com/spine-in-depth#Features> (дата обращения: 31.05.2026). – Текст. Изображение : электронные.
7. METANIT : [сайт]. – Санкт-Петербург, 2012. – URL: <https://metanit.com/sharp/wpf/22.1.php> (дата обращения: 04.06.2026). – Текст. Изображение : электронные.
8. Хабр : [сайт]. – Москва, 2003. – URL: <https://habr.com/ru/articles/434380/> (дата обращения: 04.06.2026). – Текст. Изображение : электронные.